えらぶノート 現行仕様・要件まとめ

えらぶノートの、今の仕様と設計方針をまとめたもの（2026-10-30 時点）。

今後追加する共有機能（フォローとタイムライン）の設計は sharing-plan.md を参照。

1. サービス概要

- **目的**: 化粧品や日用品を選ぶときの判断を支える、個人向けのアイテム管理アプリ。
- **コンセプト**: 自分の評価とレビューをストックし、次の買い物の判断材料にする。他人の口コミではなく、自分の感想を重視する。
- **在庫管理の方針**: 在庫数は管理しない。「なくなりそう」フラグを軽い目印として使い、リピートするか別のものを試すかを考えるきっかけにする。
- **ターゲット**
  - メイン: 日用品や化粧品を自分なりに選んで使いたい人
  - サブ: 家庭内の日用品を管理している人
- **差別化**: 口コミサイト（LIPS、@cosme）は他人のレビューが中心で、在庫管理アプリ（zaico など）は「何を持っているか」の管理が中心。このアプリは、自分の感想の記録と持ち物の状態管理を1つにまとめている。

2. 技術スタック

領域	採用技術
言語・FW	Ruby 3.2.2 / Rails 8.1
DB	PostgreSQL（本番は Neon）。主キーはすべて UUID
フロント	Tailwind CSS、Turbo、Stimulus、importmap
認証	Devise（devise-i18n）、omniauth-oauth2 による独自 LINE ストラテジ
画像	Active Storage（本番は Cloudflare R2）、image_processing（libvips）
ページネーション	Kaminari
メール	Resend（開発は letter_opener_web）
開発環境	Docker Compose
本番	Render
テスト・CI	Minitest、GitHub Actions
Lint	rubocop-rails-omakase

3. データモデル

```
users 1 ──────── * categories
users 1 ──────── * items
categories 1 ──────── * items（items.category_id は任意）
items ─── 画像1枚（Active Storage）
```

users

カラム	備考
email	必須、一意、形式チェックあり
name	必須、255文字以内、重複可
encrypted_password	Devise 管理
line_user_id	一意。LINE 連携済みの判定に使う
line_access_token	LINE のアクセストークン
reset_password_token など	Devise 管理

- **特殊アカウント（メールドメインで判定）**
  - LINE 登録: `@line.invalid` の仮メールアドレスを使う（`placeholder_email?`）
  - ゲスト: `@guest.invalid`（`guest?`）
- **削除時**: `items` と `categories` は `dependent: :destroy` で一緒に削除される。

categories

カラム	備考
user_id	必須
name	必須、20文字以内、ユーザー内で一意

- カテゴリを削除すると、紐づく items の `category_id` は `nullify` される。

items

カラム	型	備考
name	string	必須、100文字以内
brand_name	string	100文字以内
category_id	uuid	任意
price	integer	0以上の整数、全角数字は自動で半角に変換

カラム	型	備考
capacity / capacity_unit	decimal(8,2) / string	容量は 0 より大きい値。単位は `ml` `g` `kg` `枚` `個`
memo	text	登録時のメモ
rating	integer	1〜5
review	text	レビューを書く場合は `rating` が必須
favorite	boolean	よく使うもの
low_stock_flagged	boolean	なくなりそう
archived	boolean	手放した
image	Active Storage	JPEG/PNG のみ、10MB 以下。variant は thumbnail（160px）と preview（512px）、どちらも webp

4. 機能要件

4.1 認証・アカウント

- メールアドレスとパスワードによる新規登録、ログイン、パスワードリセット（Devise）
- **LINE ログイン**（OmniAuth）
  - 連携済みの LINE アカウントならログイン、未連携なら新規登録する
  - ログイン中に LINE でログインすると、今のアカウントに LINE を連携する。その LINE がすでに別のアカウントと連携している場合は、連携しない
  - アカウント編集画面から連携を解除できる（`DELETE /account/line`）
- **ゲストログイン**
  - 登録なしで試せる
  - ログインのたびに一時アカウントを新規作成する
  - カテゴリ5件とアイテム16件のサンプルデータを自動投入する。中身は `config/guest_sample_data.yml` に書いている
  - サンプルは、よく使う / なくなりそう / 高評価 / 低評価 / レビュー未記入 / 手放した、の各状態を網羅している
  - ゲストはアカウント情報を編集できない（メールアドレスを変えると、ゲストの判定と定期削除から外れてしまうため）
  - 作成から24時間以上経過したゲストは `guest:cleanup` タスク（`lib/tasks/guest.rake`）で削除する
- 静的ページ: トップ、利用規約、プライバシーポリシー、お問い合わせ（Google フォームへの導線）
- 認証は Devise の `authenticate_user!` による。アイテム関連の操作はすべて `current_user` のデータに限定する。

4.2 ホーム（`/home`）

セクション	条件	件数上限
なくなりそう	手放していない、かつ `low_stock_flagged`	4件（総数も表示）
レビュー未記入	手放していない、かつ `rating` が空	6件（総数も表示）
よかったもの	手放していない、かつ評価 4〜5	3件、評価の高い順
イマイチだったもの	手放していない、かつ評価 1〜2	3件、評価の低い順

4.3 持ち物一覧（`/items`）

- **表示範囲**: すべて（アーカイブ以外）/ 手放したもの
- **属性タグ（複数選択可）**: よく使うもの / なくなりそう / レビュー未記入
- **その他の絞り込み**: アイテム名の部分一致検索（ILIKE）、カテゴリ（「未分類」も指定可）
- **並び順**: 作成日の新しい順
- **ページネーション**: Kaminari
- **検索のオートコンプリート**: `GET /items/autocomplete`。2文字以上で、アイテム名を最大10件 JSON で返す。

4.4 アイテム登録・編集

- **入力項目**: 名前、ブランド名、カテゴリ、価格、容量と単位、画像、メモ。登録時に評価とレビューも入力できる。
- **カテゴリ**
  - 既存カテゴリから選ぶ
  - 新規カテゴリ名を入力して、その場で作成する（同名があれば再利用する）
  - カテゴリなしにする
- **画像**: 編集画面から単独で削除できる（`DELETE /items/:id/image`）。
- **物理削除**: 詳細画面の一番下に、目立たない「削除する」リンクがある（確認あり）。感想も消えてしまうため、基本は「手放す」で運用する方針。

4.5 アイテム詳細（`/items/:id`）

- カテゴリ、ブランド名、評価・感想を表示する。
- **容量あたりのコスト** は「価格 ÷ 容量」で、小数第1位まで。価格か容量が未入力なら表示しない。
- 画面上の操作:
  - よく使うもの（`favorite`）のトグル
  - なくなりそう（`low_stock_flagged`）のトグル。外すときだけ確認を出す（押し間違いで外すと、買い忘れにつながるため）
  - 感想の編集
  - 手放す
  - 削除する（物理削除。感想も消える）

- フラグを切り替えると、画面の上にメッセージを出す。
- 手放したアイテムでは、フラグのトグルと「手放す」を出さず、代わりに「リストに戻す」を出す。

4.6 感想入力・振り返り

- **感想入力（`/items/:id/edit_review`）**
  - 評価とレビューだけの専用画面
  - 何度でも上書きできる
  - アイテム編集の `update` を使わないのは、カテゴリが更新されてしまうため
- **感想一覧（`/items/reviews`）**
  - 評価が付いているアイテムが対象
  - 星評価、名前、カテゴリで絞り込める
  - 更新日の新しい順

4.7 手放す（アーカイブ）

- 削除ではなく `archived: true` にするので、感想は残る。
- 手放すと `favorite` と `low_stock_flagged` は自動でオフになる。
- 手放す前に確認画面（`confirm_archive`）を挟む。
- 「手放したもの」タブから戻せる（`unarchive`）。戻したときも `favorite` はオフのまま。
- 手放したアイテムは、フラグの切り替えと手放す操作ができない。画面でボタンを隠すだけでなく、controller（`reject_archived_item`）でも拒否する。

5. 設計方針・ルール

- **状態は `items` が直接持つ**
  - 評価やレビューの履歴テーブルは持たず、`rating` と `review` は常に上書きする。
  - 使用サイクルごとの履歴も持たない。
  - 当初は使用開始・終了を履歴で記録する設計だったが、管理が煩雑だったため、シンプルな設計に作り直した。
- 主キーは UUID。新規テーブルも `id: :uuid, default: -> { "gen_random_uuid()" }` の形式に合わせる。
- 変数名・メソッド名・カラム名は英語、画面文言とコメントは日本語。
- controller は薄くし、判定ロジックは model に寄せる。
- 入力の正規化として、価格と容量は全角数字（`０-９．`）を半角に変換する。

6. 画面・ルーティング一覧

画面	パス
トップ	`/`
ホーム	`/home`
アイテム一覧	`GET /items`

画面	パス
アイテム登録	`/items/new`
アイテム詳細	`/items/:id`
アイテム編集	`/items/:id/edit`
感想入力	`/items/:id/edit_review`
感想一覧	`/items/reviews`
手放す確認	`/items/:id/confirm_archive`
ログイン、新規登録	Devise（`users/...`）
ゲストログイン	`POST /users/guest_sign_in`
規約、プライバシー、お問い合わせ	`/terms`、`/privacy`、`/contact`

アクション系（PATCH / DELETE）：`toggle_favorite`、`toggle_low_stock`、`archive`、`unarchive`、`update_review`、`image` 削除

7. 今後の実装予定（README 記載）

- 購入リマインド通知
- お気に入りアイテムの公開、SNS 共有
- バーコードスキャンによる登録の簡略化

8. 既知の課題

- `line_access_token` **の保存**: LINE ログイン時に users テーブルへ保存しているが、保存したあとどこからも使っていない。使わないデータは持たないほうが安全なので、カラムごと削除する予定（#382）。README の ER 図にこのカラムがないのも、使っていないため。